

JavaScript Basics — Condensed Notes (Apna College)

1. console.log()

Writes (logs) a message to the browser console.

```
console.log("Apna College");
console.log(1234);
console.log(2+2);           // 4
console.log("Apna", "College", 123); // multiple args, space-separated
```

Tip: console is part of the **browser** (not JS itself) — it's a tool JS uses to talk to the browser.

2. Linking a JS File to HTML

```
<script src="app.js"></script>
```

Place inside <head> or just before </body>. Placing at the end of <body> is preferred so HTML loads first.

3. Variables (let)

```
let a = 10;
let b = 5;
console.log("sum is : ", a + b); // sum is : 15
```

Trick: // starts a single-line comment — great for disabling a line while testing.

4. Template Literals

Embed expressions inside a string using backticks ` and \${ }.

```
let a = 5, b = 10;
console.log(`Your pay ${a + b} rupees`); // Your pay 15 rupees
// Old way (string concatenation with +);
console.log("Price is", a+b, "rupees");
```

Trick: Use backticks (`) not quotes — remember "\$ { }" back ticks" together = template literal syntax.

5. Operators in JS

- **Arithmetic:** + - * / % ** (%=modulus/remainder, **=exponent)
- **Unary:** ++ (increment), -- (decrement) → e.g. a++, b--
- **Assignment:** = += -= *= /= %=
- **Comparison:** see below
- **Logical:** see below

6. Comparison Operators

Op	Meaning
>	Greater than
>=	Greater than or equal to
<	Lesser than
<=	Lesser than or equal to
==	compares value only (loose)
===	compares type & value (strict)
!=	not equal (loose)

```
"123" == 123    // true
"123" === 123   // false (diff types)
1 == '1'        // true
0 == ' '        // true
0 == false      // true
null == undefined // true, but === false
```

Best practice: Always prefer === / !== over ==/!= to avoid unexpected type coercion bugs.

7. Comparing Non-Numbers (Strings)

JS compares strings using their **Unicode** value (a unique code per character).

```
'a' -> 61(dec) ... 'A' -> 41(dec) ...
// lowercase letters have HIGHER unicode than uppercase
'a' > 'A' // true
'a' > 'b' // false
'b' < 'c' // true
'*' < 'g' // false
```

Trick: Ordering — digits < UPPERCASE < lowercase in Unicode. a<b<c... and A<B<C... each in order.

8. Conditional Statements — Overview

Types: if-else, nested if-else, switch. A **condition** evaluates to true or false; code inside { } runs only if true.

9. if Statement

```
// code before if
if (some condition) {
  // do something (runs only if condition is true)
}
// code after if
```

```
let age = 19;
if (age >= 18) {
  console.log("you can vote");
  console.log("you can drive");
}
```

10. else / else if

```
if (condition1) { // DO SOMETHING }
else { // DO SOMETHING ELSE }

// multiple conditions chained:
if (condition1) { ... }
else if (condition2) { ... }
else if (condition3) { ... }
```

```
let age = 17;
if (age >= 18) { console.log("you can vote"); }
else { console.log("you cannot vote"); }
```

Best practice: Prefer if / else if / else chains over multiple separate if blocks when conditions are mutually exclusive — cleaner & avoids unnecessary checks once a match is found.

11. Practice: Traffic Light System

```
let color = "red";
if (color === "red") { console.log("Stop!"); }
else if (color === "yellow") { console.log("slow down"); }
else if (color === "green") { console.log("go"); }
else { console.log("traffic light is broken"); }
```

12. Practice: Popcorn Pricing (nested if)

```
let size = "XL";
if (size === "XL") { console.log("Rs. 250"); }
else if (size === "L") { console.log("Rs. 200"); }
else if (size === "M") { console.log("Rs. 100"); }
else { console.log("Rs. 50"); }

// Nested example: Pass/Fail + Grade
if (marks >= 33) {
  console.log("Pass");
  if (marks >= 80) { console.log("Grade: O"); }
  else { console.log("Grade: A"); }
} else { console.log("Better luck next time!"); }
```

13. Logical Operators

Combine multiple expressions/conditions.

Op	Name	Rule
&&	AND	true only if BOTH sides true
	OR	true if EITHER side true
!	NOT	flips true↔false

```
true && true    // true
true && false   // false
false && true    // false
false && false  // false

if ((marks > 33 && marks <= 80) || !false) { console.log("pass"); }
```

Trick: "&& needs BOTH, || needs ONE" — think of && as strict, || as lenient.

14. Practice Qs (Logical Operators)

Q: A "good string" starts with 'a' AND has length > 3.

```
str[0] === 'a' // checks first character (index 0)
str.length > 3 // checks length
if ((str[0] === 'a') && (str.length > 3)) { console.log("Good"); }
else { ... }
```

Q: Predict output — num = 12:

```
if ((num%3 === 0) && ((num+1 == 15) || (num-1 == 11))) {
  console.log("safe");
} else { console.log("unsafe"); }
// 12%3===0 → true (T); 12+1=13≠15 (F); 12-1=11==11 (T)
// T && (F || T) = T && T = true → Output: "safe"
```

15. Truthy & Falsy Values

Every value in JS is treated as true or false in a boolean context (e.g. inside an if condition).

Falsy values (memorize all 7): false, 0, -0, 0n (BigInt), "" (empty string), null, undefined, NaN **Truthy values:** everything else (e.g. non-empty strings, any nonzero number, objects, arrays).

```
let str = " "; // non-empty (has a space) → truthy
if (str) { console.log("string is not empty"); } // runs

let num = -10; // nonzero → truthy
if (num) { console.log("num is not equal to 0"); } // runs
```

Trick: Only 0, empty string, null, undefined, NaN, and false are falsy — a "space" string, negative numbers, and non-empty arrays are ALL truthy!

16. Switch Statement

Used when comparing one variable against several fixed values (cleaner than long if/else-if chains).

```
let color = "red";
switch(color) {
  case "red":
    console.log("stop");
    break;
  case "yellow":
    console.log("slow down");
    break;
  case "green":
    console.log("go");
    break;
  default:
    console.log("light is broken");
}
```

Critical: Never forget break; after each case — without it, execution "falls through" into the next case(s) automatically. default runs when no case matches (like a final else).

17. Practice: Switch — Day of Week

```
let day = 7;
switch (day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break;
  case 3: console.log("Wednesday"); break;
  case 4: console.log("Thursday"); break;
  case 5: console.log("Friday"); break;
  case 6: console.log("Saturday"); break;
  case 7: console.log("Sunday, fun day!"); break;
  default: console.log("wrong day!");
}
```

18. Alert & Prompt (Browser Dialogs)

```
alert("something is wrong!"); // shows a popup message on the page
prompt("please enter your roll no."); // popup asking user for input

let firstName = prompt("enter your name : ");
console.log(firstName); // stores/prints whatever user typed
```

Trick: prompt() ALWAYS returns a string (even if user types a number) — convert with Number() if doing math on it.

19. Console Methods: log / error / warn

```
console.log("this is a simple log"); // normal text, white
console.error("this is an error msg"); // red, shows a stack trace
console.warn("this is a warning msg"); // yellow/orange warning icon
```

Trick: Use console.error/warn (not just log) while debugging — they're color-coded and easier to spot in a busy console.

Practice Questions — JS (Part 2)

- Create a number variable num. Print "good" if divisible by 10, else "bad".
- Take user's name & age via prompt(). Alert back: "name is age years old" using template literals.
- Switch statement to print the months in a quarter (Q1: Jan, Feb, Mar; Q2: Apr, May, Jun; Q3: Jul, Aug, Sep; Q4: Oct, Nov, Dec) — use the quarter number as the case value.
- A string is a "golden string" if it starts with 'A' or 'a' AND has length > 5. Print if a given string is golden or not.
- Write a program to find the largest of 3 numbers.
- (Bonus)** Check if 2 numbers have the same last digit (e.g. 32 and 47852 both end in 2) — hint: use % 10.